FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DISEÑO E IMPLEMENTACIÓN DEL SISTEMA DE GESTIÓN DE SOFTWARE PARA LA BODEGA CRUZ DE CHALPÓN

Autores:

CASTREJON RODRIGUEZ, Harold Cesar (100 %)
CENTURIÓN VIGO, José Wilfredo (100 %)
FRIAS TORRES, Luis Miguel (100 %)
MENDOZA TERRONES, Andres Jhonel (100 %)
OCAS MUÑOZ, Clemente Daniel (100 %)

Curso:

Calidad y Pruebas de Software

Docente:

URIARTE CORTEGANA, Kenny Jhoel

Cajamarca – Perú

2026 – 1

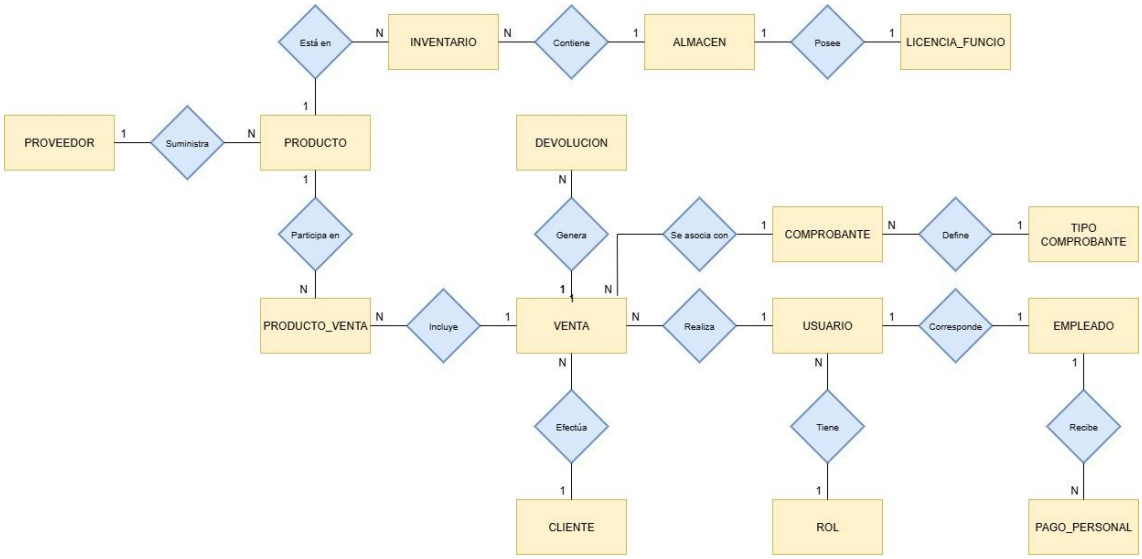
1. **Características técnicas del producto o Software a desarrollar**

El sistema está diseñado para optimizar los procesos administrativos y operativos de una bodega familiar. Sus principales características técnicas son:

- **Arquitectura:** Monolítica organizada en capas (Presentación, Lógica de Negocio y Acceso a Datos) bajo el patrón MVC (Modelo-Vista-Controlador).
- **Lenguaje de Programación:** C# utilizando la plataforma .NET.
- **Entorno de Desarrollo (IDE):** Visual Studio 2022.
- **Base de Datos:** SQL Server LocalDB (o SQL Server Express) para la persistencia de datos.
- **Atributos de Calidad (ISO/IEC 25010):** Se priorizaron la usabilidad (interfaz intuitiva para personal no técnico), seguridad (cifrado y control de accesos), mantenibilidad (módulos aislados) y disponibilidad (99.5% en horario comercial).
- **Despliegue:** Local (on-premise) para asegurar la operatividad ante conexiones inestables en zonas rurales.

2. **Diagrama de base de datos**

- Modelo conceptual de datos



- Modelo lógico

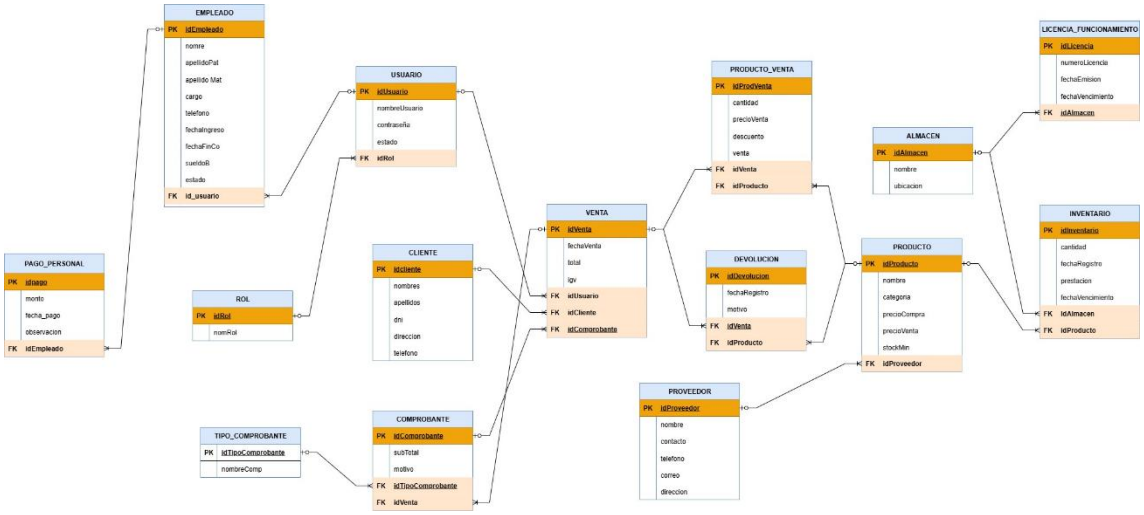
- Almacen.cs
  - Almacen
    - IdAlmacen : int
    - Nombre : string
    - Ubicacion : string
    - Productos : ICollection<Producto>
- Cliente.cs
  - Cliente
    - IdCliente : int
    - Nombres : string
    - Apellidos : string
    - DNI : int
    - Direccion : string
    - Telefono : int
    - Ventas : ICollection<Venta>
- Comprobante.cs
  - Comprobante
    - IdComprobante : int
    - SubTotal : decimal
    - Motivo : string
    - VentaId : int
    - Venta : Venta
    - ProductoId : int
    - Producto : Producto
    - IdTipoComprobante : int
    - TipoComprobante : TipoComprobante
- Devolucion.cs
  - Devolucion
    - IdDevolucion : int
    - FechaRegistro : DateTime
    - Motivo : string
    - VentaId : int
    - Venta : Venta
    - ProductoId : int
    - Producto : Producto

- Devolucion.cs
  - Devolucion
    - IdDevolucion : int
    - FechaRegistro : DateTime
    - Motivo : string
    - VentaId : int
    - Venta : Venta
    - ProductoId : int
    - Producto : Producto

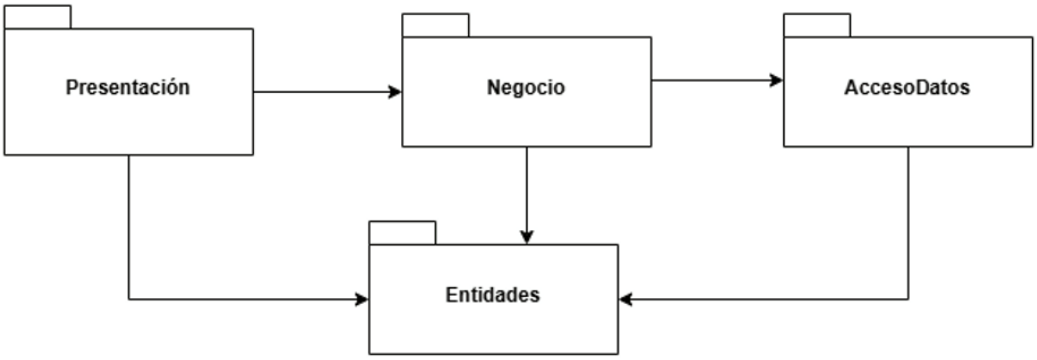
- Empleado.cs
  - Empleado
    - IdEmpleado : int
    - Nombre : string
    - ApellidoPat : string
    - ApellidoMat : string
    - Cargo : string
    - Telefono : int
    - FechaIngreso : DateOnly
    - FechaFinCo : DateOnly
    - SueldoB : decimal
    - Estado : bool
    - Usuarios : ICollection<Usuario>
    - Pagos : ICollection<Pago_Personal>
    - licencia_Funcionamientos : ICollection<Licencia_Funcionamiento>
- ErrorViewModel.cs
- Inventario.cs
  - Inventario
    - IdInventario : int
    - Cantidad : int
    - FechaRegistro : DateTime
    - Presentacion : string
    - FechaVencimiento : DateOnly
    - ProductoId : int
    - Producto : Producto
- Licencia_Funcionamiento.cs
  - Licencia_Funcionamiento
    - IdLicencia : int
    - NumeroLicencia : string
    - FechaEmision : DateTime
    - FechaVencimiento : DateTime
    - EmpleadoId : int
    - Empleado : Empleado

```
Proveedor.cs
    Proveedor
        IdProveedor : int
        Nombre : string
        Contacto : string
        Telefono : int
        Correo : string
        Direccion : string
        Productos : ICollection<Producto>
Rol.cs
    Rol
        IdRol : int
        NombreRol : string
        Usuarios : ICollection<Usuario>
TipoComprobante.cs
    TipoComprobante
        IdTipoComprobante : int
        NombreComp : string
        Comprobantes : ICollection<Comprobante>
Usuario.cs
    Usuario
        IdUsuario : int
        NombreUsuario : string
        Contraseña : string
        Estado : bool
        RolId : int
        Rol : Rol
        EmpleadoId : int
        Empleado : Empleado
        Ventas : ICollection<Venta>
Venta.cs
    Venta
        IdVenta : int
        FechaVenta : DateTime
        Total : decimal
        igv : decimal
        ClienteId : int
        Cliente : Cliente
        UsuarioId : int
        Usuario : Usuario
        ProductoVentas : ICollection<ProductoVenta>
        Comprobantes : ICollection<Comprobante>
        Devoluciones : ICollection<Devolucion>
```

❖ Modelo Físico



3. Diagrama de paquetes (Modelo Arquitectónico)



4. Resumen de Modelo OWASP y top 10 – de los últimos años

El **Modelo OWASP** (Open Web Application Security Project) es un marco de trabajo de una organización sin fines de lucro que proporciona recursos, herramientas y estándares para mejorar la seguridad del software a nivel global. Su proyecto más influyente es el **OWASP Top 10**, una lista que identifica los riesgos de seguridad más críticos para las aplicaciones web, basada en datos reales de miles de vulnerabilidades reportadas.

4.1. Control de Acceso Quebrado: Ocurre cuando las restricciones sobre lo que los usuarios autenticados pueden hacer no se aplican correctamente.

Ejemplo: Un usuario puede acceder a la cuenta de otro simplemente cambiando el ID en la URL, o un usuario común accede a funciones de administrador.

4.2. Fallos Criptográficos: Antes llamado "Exposición de Datos Sensibles". Se enfoca en la protección de datos en reposo y en tránsito.

Ejemplo: Guardar contraseñas en texto plano, usar algoritmos de cifrado obsoletos (como MD5 o SHA1) o no usar certificados TLS (HTTPS) actualizados.

4.3. Inyección: Ocurre cuando se envían datos no confiables a un intérprete como parte de un comando o consulta.

Ejemplo: Inyección SQL (acceso a bases de datos), inyección de comandos de SO o XSS (Cross-Site Scripting). Los datos del usuario "engañan" a la aplicación para ejecutar órdenes no deseadas.

4.4. Diseño Inseguro: Es una categoría nueva que se centra en los riesgos relacionados con fallos de diseño y arquitectura.

Ejemplo: Una aplicación que no tiene una función de "recuperación de contraseña" segura desde su concepción. No se soluciona programando mejor, sino diseñando mejor el flujo del negocio.

4.5. Configuraciones de Seguridad Incorrecta: Sucede cuando los ajustes de seguridad no se definen, se implementan mal o se dejan los valores por defecto.

Ejemplo: Dejar activadas funciones innecesarias, cuentas con contraseñas de fábrica o mostrar mensajes de error detallados que revelan información del servidor a los atacantes.

4.6. Componentes vulnerables y Obsoletos: Se refiere a aplicaciones que utilizan bibliotecas, frameworks o dependencias que tienen vulnerabilidades conocidas o ya no reciben soporte.

Ejemplo: Usar una versión antigua de WordPress o una librería de JavaScript que ya tiene un exploit publicado en internet.

4.7. Fallos de Identificación y Autenticación: Ocurre cuando las funciones relacionadas con el registro, inicio de sesión y gestión de sesiones permiten que un atacante suplante identidades.

Ejemplo: Permitir ataques de fuerza bruta (sin límite de intentos), permitir contraseñas muy débiles o no invalidar la sesión tras cerrar el navegador.

4.8. Fallos de Integridad de Software y Datos: Se centra en el código y los datos que se asumen como seguros sin verificar su integridad.

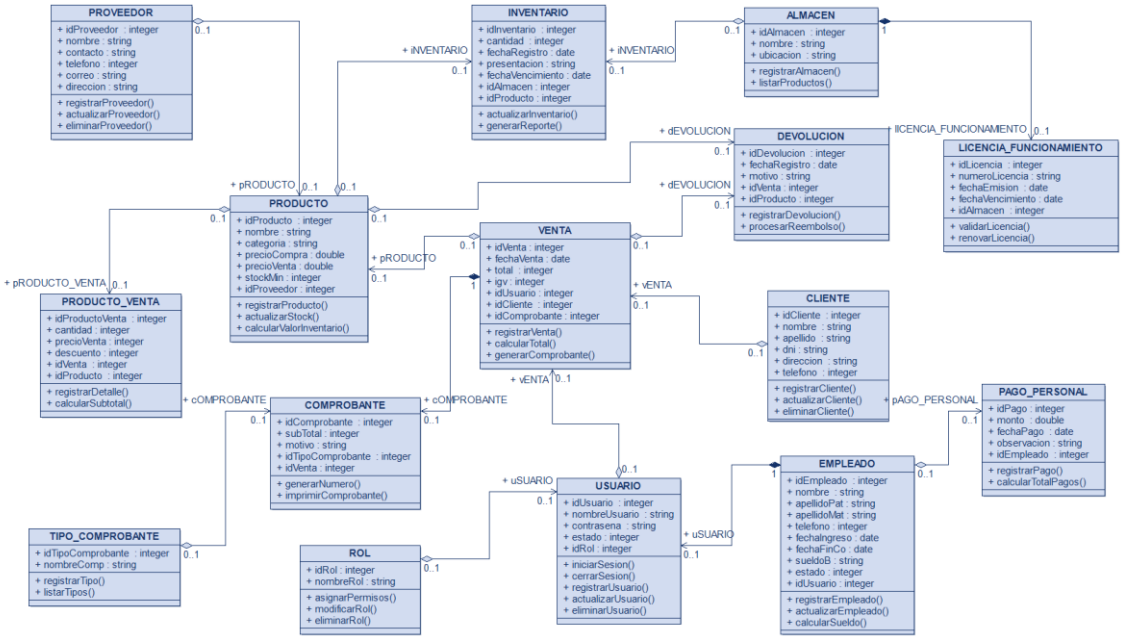
Ejemplo: Actualizaciones automáticas de software que no están firmadas digitalmente, permitiendo que un atacante inyecte código malicioso en el proceso de actualización.

4.9. Fallos de Registros y Monitoreo: Es la incapacidad de detectar y responder a brechas de seguridad en tiempo real.

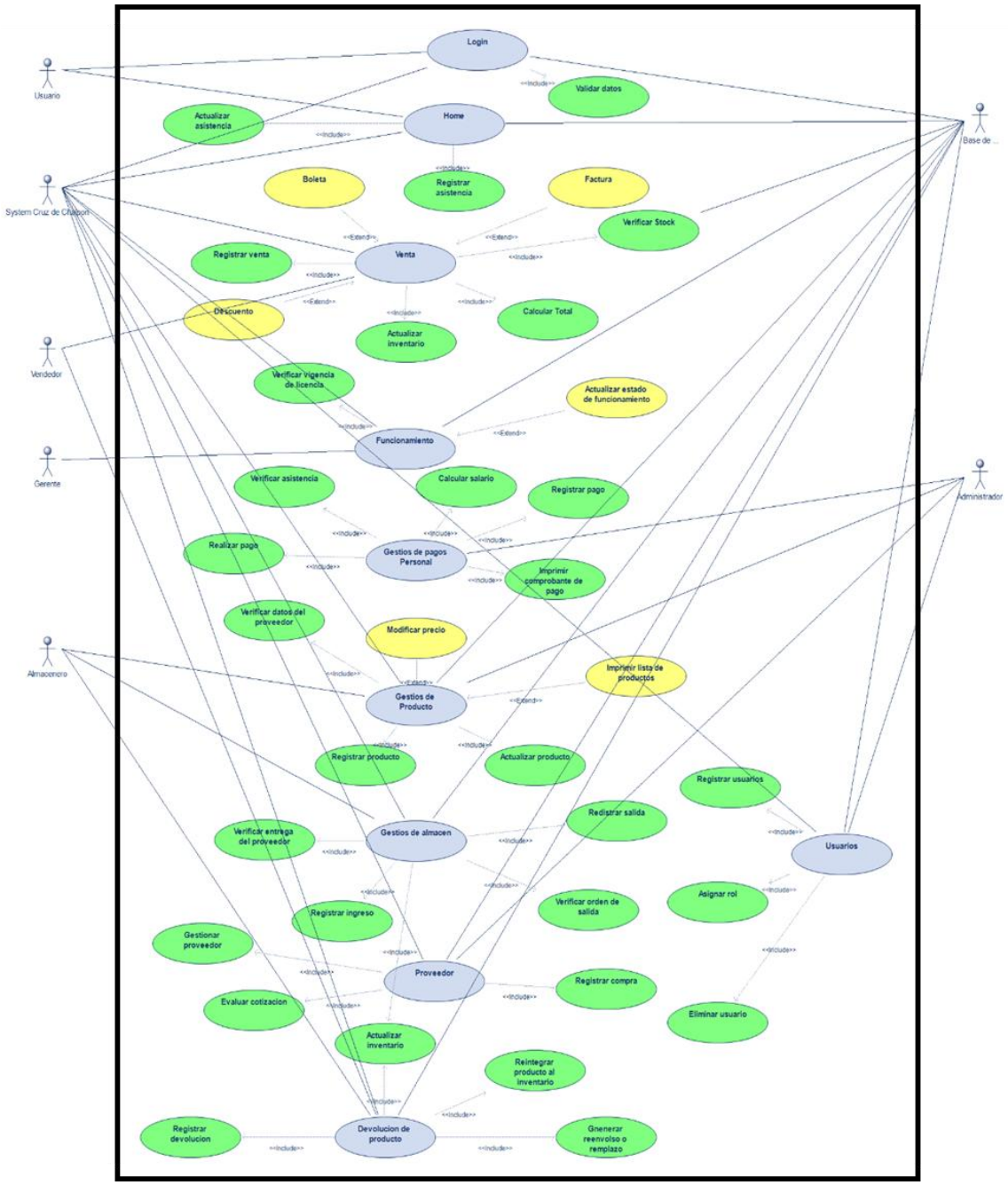
Ejemplo: si un atacante intenta entrar 10,000 veces a una cuenta y el sistema no genera una alerta o no guarda un registro (log), el ataque pasa desapercibido hasta que es demasiado tarde.

4.10. **Falsificación de Solicitudes del Lado del Servidor (SSRF):** Ocurre cuando una aplicación web recupera un recurso remoto sin validar la URL proporcionada por el usuario.
Ejemplo: Un atacante obliga a la aplicación a hacer una petición HTTP a una dirección interna de la empresa (como un panel de control privado) que no debería ser accesible desde el exterior.

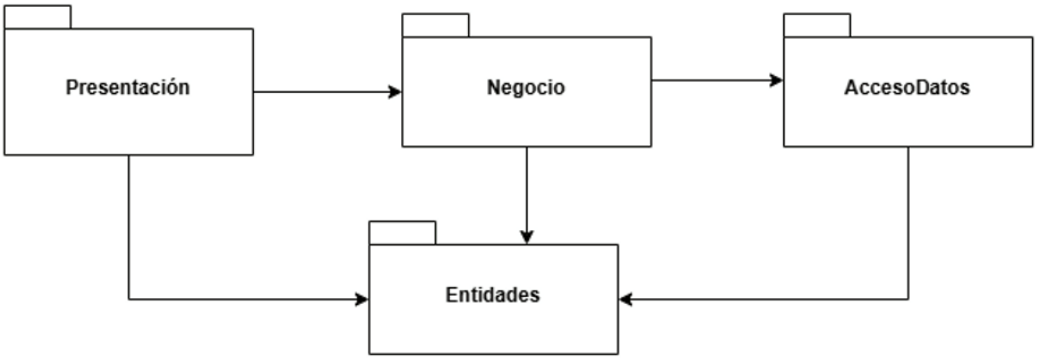
5. Diagrama de clases



6. Casos de Uso



7. Diagrama de paquetes de la arquitectura



8. Requerimientos de Software y criterios de aceptación

8.1. Requerimientos Funcionales

Nombre	Descripción
RF – 01	➢ El sistema web debe permitir la gestión de ventas, calcular el total y emitir un comprobante.
RF – 02	➢ El sistema web debe permitir registrar, modificar y eliminar información de los empleados.
RF – 03	➢ El sistema web debe alertar si licencia de funcionamiento está vigente o próxima a vencer.
RF – 04	➢ El sistema web debe brindar una capacitación de uso. ➢ El sistema web debe mostrar reporte a gerencia.
RF – 05	➢ El sistema web debe calcular y registrar pagos al personal, incluyendo bonificaciones y descuentos.
RF – 06	➢ El sistema web debe permitir generar reporte de lista de productos con bajo stock ➢ El sistema web debe listar a todos los proveedores confiables.
RF – 07	➢ El sistema web debe mostrar stock actual y generar alertas por bajo stock. ➢ El sistema web debe permitir solicitar productos a almacén.
RF – 08	➢ El sistema web debe permitir registrar la recepción de mercadería con fecha, proveedor y productos. ➢ El sistema web debe mostrar reporte de recepción de mercadería al administrador.

RF – 09	➢ El sistema web debe permitir la gestión del inventario general (consultar, actualizar y eliminar productos).
RF – 10	➢ El sistema debe permitir registrar el lugar, condiciones y estado del almacenamiento del producto.
RF – 11	➢ El sistema web debe permitir registrar una solicitud de pedido, y ver su estado (aceptado, rechazado, pendiente, atendido).
RF – 12	➢ El sistema web debe permitir gestionar la salida de mercadería por venta.
RF – 13	➢ El sistema web debe permitir registrar las devoluciones de productos por parte del cliente.
RF – 14	➢ El sistema web debe de contar con su libro de reclamaciones, en donde se pueda registrar y consultar reclamos ingresados.

8.2. Requerimientos No Funciones

Nombre	Descripción
RNF – 01 Usabilidad	➢ El sistema web debe ser accesible desde una interfaz amigable y sencilla para usuarios sin formación técnica
RNF – 02 Rendimiento	➢ El sistema web debe procesar operaciones de ventas y consultas de stock en menos de 8 segundo.
RNF – 03 Seguridad	➢ El sistema web debe contar con validación de usuarios y contraseñas para acceder a las funcionalidades.
RNF – 04 Portabilidad	➢ El sistema web debe poder ser utilizado en navegadores (Microsoft Edge, Google Crome, Opera, Mozilla Firefox).
RNF – 05 Mantenibilidad	➢ El sistema web debe permitir actualizaciones sin afectar los datos registrados.
RNF – 06 Fiabilidad	➢ El sistema web debe garantizar que los datos registrados no se pierdan ante un fallo del equipo.

RNF – 07 Escalabilidad	➢ El sistema web debe estar diseñado para crecer e integrar nuevas funciones a futuro.
RNF – 08 Respaldo de Datos	➢ El sistema web debe realizar copias de seguridad automatizada todos los días de tipo LOG y una copia de seguridad los fines de mes y Domingos de tipo FULL.

8.3. Historia de Usuario y Criterios de Aceptación

- **Historia 1: Visualizar reportes de ventas e inventario**
  - **Como:** Gerente
  - **Quiero:** visualizar reportes de ventas e inventario
  - **Para:** tomar decisiones estratégicas y controlar el rendimiento del negocio
- **Criterios de Aceptación:**
  - El sistema debe mostrar un reporte de ventas por día, semana o mes.
  - El sistema debe mostrar el total de ingresos generados en el periodo seleccionado.
  - El sistema debe listar los productos más vendidos.
  - El sistema debe mostrar el estado actual del inventario (stock disponible).
  - El sistema debe permitir filtrar la información por fechas.
  - El sistema debe permitir visualizar los reportes de forma clara y ordenada.
- **Historia 2: Registrar y gestionar proveedores**
  - **Como:** administrador
  - **Quiero:** registrar y gestionar proveedores
  - **Para:** mantener un control organizado de los abastecimientos de productos
- **Criterios de Aceptación:**
  - El sistema debe permitir registrar un proveedor con datos como nombre, teléfono y dirección.
  - El sistema debe permitir editar la información de un proveedor existente.

        - El sistema debe permitir eliminar proveedores.
        - El sistema debe listar todos los proveedores registrados.
        - El sistema debe validar que no se registren proveedores con datos vacíos.
        - El sistema debe mostrar mensajes de confirmación al registrar, editar o eliminar un proveedor.
    - **Historia 3: Registrar ventas de productos**
        - **Como:** vendedor
        - **Quiero:** registrar ventas de productos
        - **Para:** generar comprobantes y concretar la venta
    - **Criterios de Aceptación**
        - El sistema debe permitir seleccionar uno o más productos.
        - El sistema debe permitir ingresar la cantidad por producto.
        - El sistema debe calcular automáticamente el total de la venta.
        - El sistema debe validar que exista stock suficiente antes de confirmar la venta.
        - El sistema debe actualizar el stock después de la venta.
        - El sistema debe generar boleta o factura al finalizar la venta.
    - **Historia 4: Gestión de personal y Pagos**
        - **Como:** administrador
        - **Quiero:** Gestionar los datos de los empleados y calcular sus pagos mensuales.
        - **Para:** Llevar un control administrativo eficiente de la planilla.
    - **Criterios de Aceptación:**
        - Debe permitir registrar, editar y dar de baja (borrado lógico) a empleados.
        - El módulo de pagos debe permitir ingresar bonos y descuentos.
        - El sistema debe mostrar el neto a pagar calculado automáticamente antes de guardar

- **Historia 5: Recursos Humanos y Seguridad**
  - **Como:** Administrador
  - **Quiero:** Gestionar los datos de los empleados y calcular sus pagos mensuales
  - **Para:** Llevar un control administrativo eficiente de la planilla
- **Criterios de Aceptación:**
  - Debe permitir registrar, editar y dar de baja (borrado lógico) a empleados.
  - El módulo de pagos debe permitir ingresar bonos y descuentos.
  - El sistema debe mostrar el neto a pagar calculado automáticamente antes de guardar.
- **Historia 6: Control de Acceso (Seguridad)**
  - **Como:** Usuario del sistema
  - **Quiero:** Iniciar sesión con mis credenciales
  - **Para:** Acceder a las funciones permitidas según mi rol (Vendedor, Administrador, Almacenero)
- **Criterios de Aceptación:**
  - El sistema debe validar usuario y contraseña contra la base de datos.
  - Las contraseñas deben estar cifradas en SQL Server.
  - Si el usuario no tiene permisos para un módulo (ej. Vendedor entrando a Reportes), el sistema debe denegar el acceso.
- **Historia 7: Respaldos (Backups)**
  - **Como:** Administrador de Sistemas
  - **Quiero:** Que el sistema realice copias de seguridad automáticas
  - **Para:** Garantizar la recuperación de datos ante fallos (Fiabilidad)
- **Criterios de Aceptación:**
  - Se debe ejecutar un backup de LOG diariamente a una hora de baja carga.
  - Se debe ejecutar un backup FULL todos los domingos y fines de mes.

        - El sistema debe dejar un log de "Éxito" o "Error" tras cada ejecución.
    - **Historia 8: gestionar productos del inventario**
        - **Como:** administrador
        - **Quiero:** registrar, editar y eliminar productos
        - **Para:** mantener actualizado el inventario
    - **Criterios de Aceptación:**
        - El sistema debe permitir registrar nuevos productos con sus datos obligatorios.
        - El sistema debe permitir editar la información de productos existentes.
        - El sistema debe permitir eliminar productos del sistema.
        - El sistema debe validar que no existan campos vacíos en el registro.
        - El sistema debe mostrar mensajes de confirmación en cada operación.
    - **Historia 9: Registro de pagos a proveedores**
        - **Como:** administrador
        - **Quiero:** registrar pagos realizados a proveedores
        - **Para:** llevar un control financiero claro de las cuentas por pagar y deudas liquidadas.
    - **Criterios de Aceptación:**
        - El sistema debe permitir seleccionar un proveedor y una factura de compra pendiente.
        - El sistema debe registrar la fecha del pago, el monto y el método de pago (Efectivo, Transferencia, etc.).
        - El estado de la factura de compra debe cambiar de "Pendiente" a "Pagado" automáticamente.
    - **Historia 10: Devoluciones de Productos**
        - **Como:** Administrador
        - **Quiero:** Registrar la devolución de un producto vendido
        - **Para:** Mantener el equilibrio de caja y el estado real de los reclamos

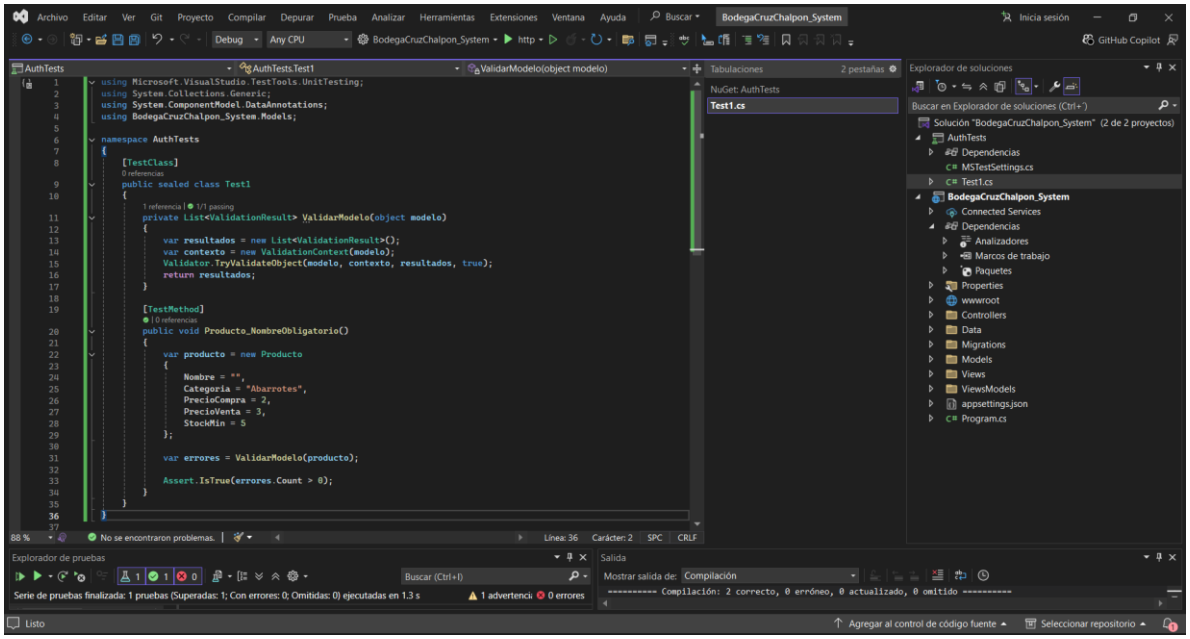
- **Criterios de Aceptación:**
  - Se debe ingresar el ID de la venta original para validar la devolución.
  - El sistema debe permitir marcar si el producto regresa al inventario o se desecha por mal estado.
  - Se debe generar un registro de "Nota de Crédito" o anulación vinculada a la venta.
- **Historia 11: Control de Stock y Alertas**
  - **Como:** Almacenero / Administrador
  - **Quiero:** Visualizar el stock actual y recibir alertas de niveles bajos
  - **Para:** Evitar la ruptura de stock y realizar pedidos a tiempo.
- **Criterios de Aceptación:**
  - El panel principal debe mostrar una lista de productos cuyo stock sea <= al stock mínimo definido.
  - El sistema debe enviar una alerta visual (color rojo/notificación) cuando un producto alcance el límite crítico.
  - Debe permitir exportar la lista de productos por agotar a formato Excel.
- **Historia 12: Recepción de Mercadería (Ingreso a Almacén)**
  - **Como:** Almacenero
  - **Quiero:** Registrar el ingreso de productos que llegan de los proveedores
  - **Para:** Incrementar el stock real y verificar que lo recibido coincida con lo pedido
- **Criterios de Aceptación:**
  - El sistema debe permitir buscar el producto por nombre o código de barras
  - El sistema debe solicitar la fecha de vencimiento y el número de lote (crítico para productos perecederos)
  - Al confirmar el ingreso, el stock en la base de datos debe sumarse automáticamente

- **Historia 13: Gestión de Clientes**
    - **Como:** Vendedor.
    - **Quiero:** Registrar y consultar datos de los clientes (DNI/RUC).
    - **Para:** Emitir comprobantes personalizados y conocer la recurrencia de compra.
- **Criterios de Aceptación:**
    - El sistema debe permitir registrar clientes con DNI/RUC, nombre/razón social y dirección.
    - Al realizar una venta, el sistema debe permitir buscar al cliente por su número de documento.
    - Debe validar que el formato del RUC (11 dígitos) o DNI (8 dígitos) sea correcto.
- **Historia 14: Configuración de Alertas de Licencias y Documentos**
    - **Como:** Administrador
    - **Quiero**: Configurar las fechas de vencimiento de la licencia de funcionamiento y certificados
    - **Para:** Recibir alertas preventivas y evitar multas por documentos vencidos
- **Criterios de Aceptación:**
    - El sistema debe permitir ingresar la fecha de vencimiento de la licencia de funcionamiento.
    - El sistema debe mostrar una alerta en el dashboard principal 30 días antes del vencimiento.
    - El estado de la licencia debe mostrarse como "Vigente" o "Próximo a vencer".
- **Historia 15:**
    - **Como:** Vendedor / Encargado de Tienda.
    - **Quiero:** Generar una solicitud de productos al almacén central.
    - **Para:** Reponer los estantes de venta cuando el stock de exhibición es bajo.

- **Criterios de Aceptación:**
  - El sistema debe permitir crear una lista de productos y cantidades solicitadas.
  - La solicitud debe quedar en estado "Pendiente" hasta que el Almacenero la apruebe o despache.
  - Debe permitir consultar el historial de solicitudes enviadas y su estado (Atendido/Rechazado).

9. Implementar pruebas unitarias

- **Prueba 1:** Verificar que el sistema no permita registrar un producto sin nombre.



- **Prueba 2:** Verifica que el sistema no permita registrar un producto sin categoría, porque Categoría es un campo obligatorio.

```
    }
    [TestMethod]
    | 0 referencias
    public void Producto_CategoriaObligatoria()
    {
        var producto = new Producto
        {
            Nombre = "Arroz",
            Categoria = "",
            PrecioCompra = 2,
            PrecioVenta = 3,
            StockMin = 5
        };

        var errores = ValidarModelo(producto);

        Assert.IsTrue(errores.Count > 0);
    }

}
```

- **Prueba 3:** Esta prueba verifica que un producto tenga todos sus datos obligatorios completos para que sea válido y no genere errores de validación.

```
    [TestMethod]
    | 0 referencias
    public void Producto_Valido_NoDebeTenerErrores()
    {
        var producto = new Producto
        {
            Nombre = "Azúcar",
            Categoria = "Abarrotes",
            PrecioCompra = 2,
            PrecioVenta = 3,
            StockMin = 5
        };

        var errores = ValidarModelo(producto);

        Assert.AreEqual(0, errores.Count);
    }
}
```

- **Prueba 4:** Esta prueba verifica que el precio de venta sea mayor que el precio de compra, para que el producto tenga una ganancia y no se venda con pérdida.

```
[TestMethod]
| 0 referencias
public void Producto_PrecioVentaMayorQuePrecioCompra()
{
    decimal precioCompra = 2;
    decimal precioVenta = 3;

    bool resultado = precioVenta > precioCompra;

    Assert.IsTrue(resultado);
}
```

- **Prueba 5:** Esta prueba verifica que el sistema no permita registrar un proveedor sin nombre, porque el campo Nombre es obligatorio en el modelo Proveedor.

```
[TestMethod]
| 0 referencias
public void Proveedor_NombreObligatorio()
{
    var proveedor = new Proveedor
    {
        Nombre = "",
        Contacto = "Juan Pérez",
        Telefono = 987654321,
        Correo = "proveedor@gmail.com",
        Direccion = "Av. Principal 123"
    };

    var errores = ValidarModelo(proveedor);

    Assert.IsTrue(errores.Count > 0);
}
```

- **Prueba 6:** Esta prueba verifica que el sistema no permita registrar un proveedor sin contacto, porque el campo Contacto es obligatorio en el modelo Proveedor.

```
[TestMethod]
0 referencias
public void Proveedor_ContactoObligatorio()
{
    var proveedor = new Proveedor
    {
        Nombre = "Distribuidora San Juan",
        Contacto = "",
        Telefono = 987654321,
        Correo = "proveedor@gmail.com",
        Direccion = "Av. Principal 123"
    };

    var errores = ValidarModelo(proveedor);

    Assert.IsTrue(errores.Count > 0);
}
```

- **Prueba 7:** Esta prueba verifica que el sistema no permita registrar un proveedor sin correo, porque el campo Correo es obligatorio en el modelo Proveedor.

```
[TestMethod]
0 referencias
public void Proveedor_CorreoObligatorio()
{
    var proveedor = new Proveedor
    {
        Nombre = "Distribuidora San Juan",
        Contacto = "Juan Pérez",
        Telefono = 987654321,
        Correo = "",
        Direccion = "Av. Principal 123"
    };

    var errores = ValidarModelo(proveedor);

    Assert.IsTrue(errores.Count > 0);
}
```

- **Prueba 8:** Esta prueba verifica que un proveedor tenga todos sus datos obligatorios completos para que sea válido y no genere errores de validación.

```
[TestMethod]
| 0 referencias
public void Proveedor_Valido_NoDebeTenerErrores()
{
    var proveedor = new Proveedor
    {
        Nombre = "Distribuidora San Juan",
        Contacto = "Juan Pérez",
        Telefono = 987654321,
        Correo = "proveedor@gmail.com",
        Direccion = "Av. Principal 123"
    };

    var errores = ValidarModelo(proveedor);

    Assert.AreEqual(0, errores.Count);
}
```

- **Prueba 9:** Esta prueba verifica que el sistema no permita registrar un usuario sin nombre de usuario, porque ese campo es obligatorio para identificar quién ingresará al sistema.

```
[TestMethod]
| 0 referencias
public void Usuario_NombreUsuarioObligatorio()
{
    var usuario = new Usuario
    {
        NombreUsuario = "",
        Contraseña = "123456",
        Estado = true,
        IdRol = 1,
        IdEmpleado = 1
    };

    var errores = ValidarModelo(usuario);

    Assert.IsTrue(errores.Count > 0);
}
```

- **Prueba 10:** Esta prueba verifica que el sistema no permita registrar un usuario sin contraseña, porque ese campo es obligatorio para poder iniciar sesión.

```
[TestMethod]
| 0 referencias
public void Usuario_ContraseñaObligatoria()
{
    var usuario = new Usuario
    {
        NombreUsuario = "admin",
        Contraseña = "",
        Estado = true,
        IdRol = 1,
        IdEmpleado = 1
    };

    var errores = ValidarModelo(usuario);

    Assert.IsTrue(errores.Count > 0);
}
```

- **Prueba 11:** Esta prueba verifica que un usuario tenga todos sus datos obligatorios completos para que sea válido y no genere errores de validación.

```
[TestMethod]
| 0 referencias
public void Usuario_Valido_NoDebeTenerErrores()
{
    var usuario = new Usuario
    {
        NombreUsuario = "admin",
        Contraseña = "123456",
        Estado = true,
        IdRol = 1,
        IdEmpleado = 1
    };

    var errores = ValidarModelo(usuario);

    Assert.AreEqual(0, errores.Count);
}
```

- **Prueba 12:** Esta prueba verifica que la contraseña del usuario no supere los 20 caracteres.

```
[TestMethod]
| 0 referencias
public void Usuario_ContraseñaNoDebePasarDe20Caracteres()
{
    var usuario = new Usuario
    {
        NombreUsuario = "admin",
        Contraseña = "12345678901234567890 1",
        Estado = true,
        IdRol = 1,
        IdEmpleado = 1
    };

    var errores = ValidarModelo(usuario);

    Assert.IsTrue(errores.Count > 0);
}
```

- **Prueba 13:** Esta prueba verifica que una contraseña muy corta no sea considerada válida, porque por seguridad debería tener como mínimo 6 caracteres.

```
[TestMethod]
| 0 referencias
public void Usuario_ContraseñaDebeTenerMinimo6Caracteres()
{
    var usuario = new Usuario
    {
        NombreUsuario = "admin",
        Contraseña = "123",
        Estado = true,
        IdRol = 1,
        IdEmpleado = 1
    };

    bool contraseñaValida = usuario.Contraseña.Length >= 6;

    Assert.IsFalse(contraseñaValida);
}
```

- **Prueba 14:** Esta prueba verifica que un usuario con Estado = false sea reconocido como inactivo, por lo tanto, no debería estar habilitado para ingresar al sistema.

```
[TestMethod]
| 0 referencias
public void Usuario_InactivoNoDebeEstarHabilitado()
{
    var usuario = new Usuario
    {
        NombreUsuario = "cajero1",
        Contraseña = "123456",
        Estado = false,
        IdRol = 2,
        IdEmpleado = 1
    };

    Assert.IsFalse(usuario.Estado);
}
```

- **Prueba 15:** Esta prueba verifica que un almacén con sus datos completos sí sea válido y no genere errores.

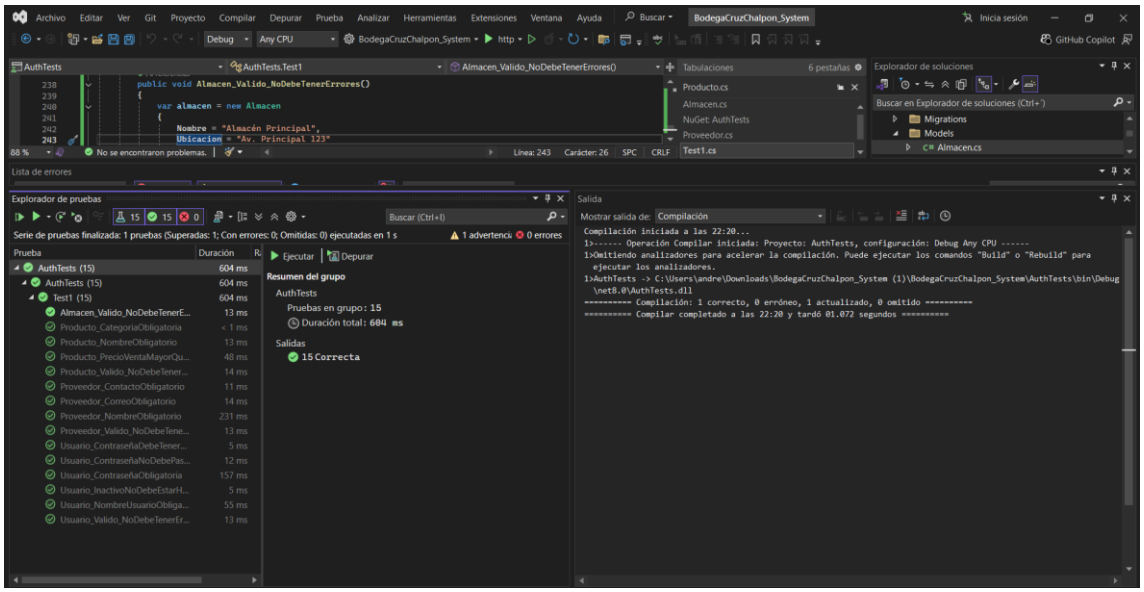
```
| 0 referencias
public void Almacen_Valido_NoDebeTenerErrores()
{
    var almacen = new Almacen
    {
        Nombre = "Almacén Principal",
        Ubicacion = "Av. Principal 123"
    };

    var errores = ValidarModelo(almacen);

    Assert.AreEqual(0, errores.Count);
}
```

10. Inspeccionar y refactorizar software

Detalle de las pruebas realizadas y los resultados obtenidos



11. Casos de pruebas

ID	Funcionalidad	Datos de Entrada	Resultado Esperado	Tipo de Prueba
CP01	Registro de Producto	Nombre: "Arroz 1kg", Precio: 3.50, Stock: 50	El sistema guarda el producto y lo muestra en la lista.	**Correcta**
CP02	Validación de Obligatorios	Nombre: "", RUC: "12345" (Proveedor)	El sistema bloquea el registro y muestra alerta: "Campo obligatorio".	**Con Error**
CP03	Límite de Contraseña	Contraseña: "123"	El sistema rechaza la clave por ser menor a 6 caracteres (seguridad).	**Caso Límite**
CP04	Formato de DNI	DNI: "12345" (solo 5 dígitos)	El sistema indica que el DNI debe tener 8 dígitos exactos.	**Con Error**
CP05	Cálculo de Stock	Venta de 10 unidades de un producto con stock de 50.	El stock final del producto debe actualizarse a 40 automáticamente.	**Correcta**
CP06	Acceso al Sistema	Usuario: "Admin", Clave: "Incorrecta123"	El sistema deniega el acceso y muestra "Credenciales inválidas".	**Con Error**
CP07	Límite de Caracteres	Nombre Usuario: "UnNombreExtremadamenteLargoMasDe20Caract"	El sistema recorta o bloquea el ingreso según el límite de la BD.	**Caso Límite**

12. Lista de control de calidad

N°	Criterio de Verificación	Cumple (Sí/No)	Observación
1	¿El sistema valida que los campos obligatorios no estén vacíos?	**Sí**	Validado en pruebas de Proveedor y Usuario.
2	¿La interfaz permite una navegación fluida entre Ventas y Almacén?	**Sí**	Según el diseño del menú principal.
3	¿Se restringe el acceso a usuarios con estado "Inactivo" (false)?	**Sí**	Verificado en la Prueba 14
4	¿Las contraseñas cumplen con el mínimo y máximo de caracteres?	**Sí**	Validado en las Pruebas 12 y 13.
5	¿El sistema responde en menos de 5 segundos al realizar búsquedas?	**Sí**	Rendimiento óptimo en entorno local.
6	¿Se visualizan correctamente los datos del Modelo Físico en la interfaz?	**Sí**	Sincronización correcta con SQL Server.
7	¿El sistema maneja errores de base de datos sin cerrarse (try-catch)?	**Sí**	Implementado en la lógica de capas (C#).
8	¿El stock se descuenta automáticamente tras una venta?	**Sí**	Lógica de negocio verificada.